Poker Club OS — אפיון טכני מלא (v0.4.0)

Nov 6, 2026 · @Netanel Mekonen

Poker Club OS למועדוני פוקר: אפליקציית שחקן, קונסולת B2B הוא אב-טיפוס מלא של פלטפורמת אחד בדפדפן, ללא שרת, עם HTML וקונסולת פלטפורמה — הכול רץ כקובץ CRM, ניהול מועדון עם מנוע פוקר וטורנירים אמיתי. המסמך מתאר למתכנת מה קיים, איך זה בנוי, איך כל חלק עובד, מה מדומה, ואיך ממשיכים לייצור.

1. מה המערכת בפועל

המוצר כולו הוא קובץ אחד, `dist/index.html` (כ-1.7MB), שרץ בדפדפן בלי שום שרת. אין API, אין בסיס נתונים מרכזי, אין חשבונות אמיתיים. ה"שרת" הוא שכבת קוד TypeScript בתוך הדף (`src/platform/*`) שמתנהגת כמו שרת: בודקת הרשאות, מאמתת קלט, משנה נתונים, כותבת ביקורת והתראות. זו ההבחנה הכי חשובה למתכנת: **כל בדיקת הרשאה היא בדיקת דמו בצד הלקוח**, לא אבטחה אמיתית.

איך זה עובד היום	נושא
אובייקט `DB` אחד ב-`localStorage` תחת המפתח `pcos.db.v7`; נשמר אוטומטית אחרי כל שינוי (debounce). מצב ניווט תחת `pcos.session.v3`	אחסון
`src/platform/seed.ts` בונה DB מלא: 3 מועדונים, ~200 משתמשים (רובם בוטים מסומנים), היסטוריית טורנירים של חודשים, CRM, ליגות, הפניות, מקרי יושרה	נתוני דמו
שמור שגילו מעל 8 שעות נזרק ונזרע מחדש, כדי שטורנירים מתוזמנים תמיד יהיו בעתיד הקרוב (`STALE_AFTER_MS` ב-`store.ts`). שינוי `SCHEMA_VERSION` ב-`seed.ts` גם מאפס	התיישנות
6 זהויות דמו שמחליפים ביניהן מהסרגל העליון: `u_daniel` (שחקן חדש), `u_player` (Alex Morgan, חבר בשני מועדונים), `u_owner` (בעלים Midnight), `u_dana` (בעלים River Kings), `u_tm` (מנהל טורנירים), `u_admin` (אדמין פלטפורמה)	זהויות
`clock.now()` ב-`core.ts` — נקודת החלפה אחת לזמן, כך שבדיקות מזיזות זמן. ה-Runtime מתקתק כל 250ms	שעון

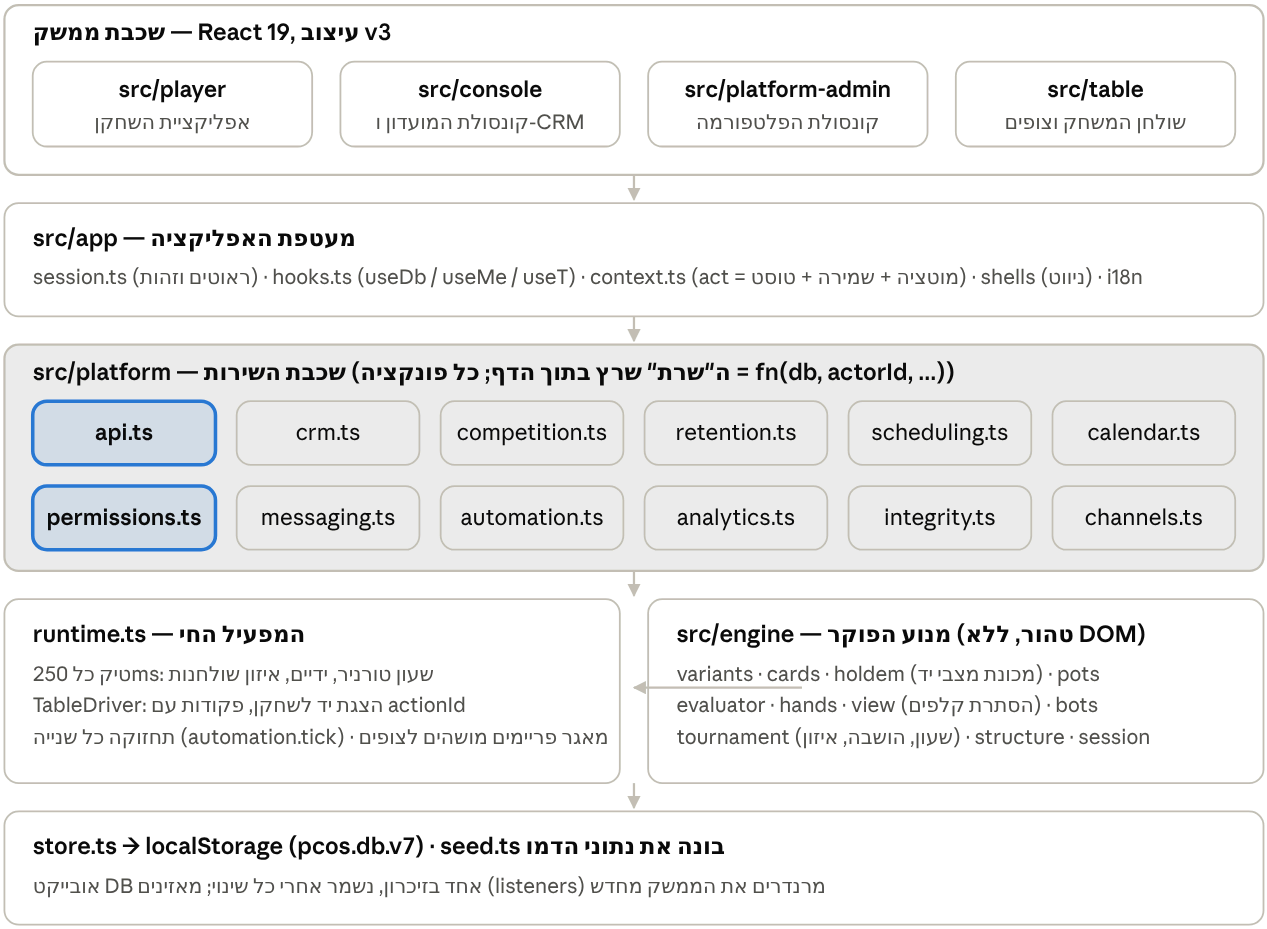
נושא	איך זה עובד היום
כסף	אין. כל הצ׳יפים והנקודות הם יחידות דמו. אין הפקדות, משיכות, שערים או סליקה בשום מקום בקוד
רשת	הבקשה היחידה היא לגופני Google; בלי רשת האפליקציה עובדת עם גופני מערכת

האפליקציה עצמה מורכבת משלוש סביבות באותו דף, שנבחרות לפי הראוט: `player` (אפליקציית השחקן), `console` (קונסולת המועדון, לפי `clubId`), `platform` (קונסולת הפלטפורמה).

2. ארכיטקטורה

חמש שכבות, תלות בכיוון אחד בלבד: הממשק קורא ומשנה נתונים רק דרך שכבת השירות, שכבת השירות מפעילה את המנוע, והכול נשמר ב-Store אחד.

כל השכבות רצות בדף אחד: ממשק → שירות → מנוע → אחסון מקומי



שכבות המערכת · 5 שכבות, כיוון אחד

הכלל שמחזיק את המבנה: מסך לעולם לא קורא ישירות מאוספים פרטיים (`accessCode` , `db.crm.*`) או מחישובי אנליטיקה ללא שומר הרשאה; בדיקת ארכיטקטורה ב- `tests/crm.test.ts` סורקת את קבצי הממשק ונכשלת על כל חריגה. זה התפר שבו יושב בעתיד ה-API: כל פונקציית שירות היא בצורה `fn(db, actorId, ...args)` , כש- `actorId` בייצור מגיע מה-session ו- `db` הופך לטרנזקציה.

סדר הייבוא בשכבת השירות (כיוון אחד, כדי שכל מודול ניתן לבדיקה לבד): `core ← analytics ← runtime ← api ← automation ← messaging ← crm` . מודולי v0.4 (`competition` , `retention` , `scheduling` , `calendar` , `migration` , `integrity` , `channels`) יושבים לצדם ונשענים על אותם שומרים.

מחזור חיי של פעולה בממשק (למשל אישור חבר): המסך קורא `act(() => api.approveMember(db, me.id, clubId, userId), { key: 'toast.key' })` מ- `context.ts` . `act` מריץ את הפונקציה, תופס `DomainError` / `ForbiddenError` ומתרגם אותם לטוסט (`err.<code>`), מסמן את ה-Store כמשונה (שמירה + רנדור מחדש לכל המאזינים) ומחזיר את התוצאה או `undefined` בכישלון.

3. טכנולוגיות, בנייה והרצה

הקוד הוא כ-44,000 שורות TypeScript/TSX ב- `src/` , בלי שום פריימוורק צד-שרת.

רכיב	בחירה	הערות למתכנת
UI	React 19 + react-icons (Lucide)	פונקציות בלבד, hooks; אין Redux — המצב הוא ה-Store עצמו
שפה	TypeScript strict, `noUnusedLocals`	`npx tsc --noEmit -p tsconfig.json` חייב להיות נקי לפני כל commit
סטיילינג	Tailwind v4 + שני גיליונות: `styles/app.css` (v0.2 מראה) ו- `styles/app-v3.css` (v0.3, מראה הנוכחי)	אותם טוקנים (`--color-bg` , `--color-surface` , `--color-accent` ...) בשני הגיליונות; הקומפוננטות אדישות למראה
בנייה	`node scripts/build.mjs --design=v2|v3` (esbuild)	מייצר `dist/index.html` (עצמאי) ו- `dist/artifact.html` (פרגמנט להטמעה). הדגל `__DESIGN__` נקבע בזמן בנייה ו- `src/app/variants.ts` בוחר shells/Home/TableStage לפי הגרסה
גרפים	ידני SVG + (סקאלות) d3 ב- `ui/charts.tsx`	Heatmap, Sparkline, StatTile, Funnel
QR	מקודד עצמי `ui/qr.ts` (byte mode, EC-M, גרסאות 1–9)	אומת בפענוח OpenCV בזמן הפיתוח
בדיקות יחידה	`node --import tsx --test tests/*.test.ts` (node:test)	החבילה המלאה ~4 דקות; `crm.test.ts` לבדו ~2.5 דק׳ — מריצים בחלקים
E2E	Playwright Chromium, `node scripts/e2e.mjs --only=A|B|C|D|E|F|S`	רץ נגד `dist/index.html` ; **אסור לבנות מחדש בזמן ריצת E2E**
תרגום	`node --import tsx scripts/i18n-check.mjs`	נכשל על מפתח חסר באחת השפות
תלויות	`node_modules` הוא symlink ל- `/opt/npm-tools/node_modules` בסביבת הפיתוח	בסביבה חדשה: `npm install` רגיל (package.json מגדיר הכול)

זרימת עבודה מומלצת לשינוי: לערוך → `tsc` → `i18n-check` → בדיקות היחידה הרלוונטיות → `build --design=v3` → מסע E2E רלוונטי → commit. אין lint נפרד; prettier מוגדר ב-package.json.

4. מודל הנתונים

כל הנתונים חיים באובייקט אחד מטיפוס `DB` (`src/platform/types.ts`, ~1,050 שורות של טיפוסים — זה מסמך הסכימה האמיתי). מזהים הם מחרוזות עם קידומת (`u_`, `c_`, `t_`, `lnk_`, `seg_`...) מונפקות על ידי `newId(db, prefix)`; נתוני דמו זרועים מקבלים סיומת `S` (`t_S12`). אוספים שנשלפים לפי מזהה הם `Record<id, T>`; רשימות שנסרקות הן מערכים.

מה זה מחזיק	טיפוס	אוסף ב-DB
שם, handle, email, גוון אוואטר (`hue`), `isBot`, `platformRole`, שפה	`User`	`users`
ערכת קלפים, ערכת שולחן, צלילים, העדפות התראות לפי סוג, פרטיות (`showProfile`), ערוצים, משחק אחראי	`UserSettings`	`settings`
שם, קוד 6 ספרות, `inviteToken`, מיתוג (`branding`: accent, logo, cover, tableTheme, titleFont, tablePresets), מדיניות הצטרפות, אזור זמן, תוכנית, סטטוס, `nextMemberNo`, קישורים חברתיים	`Club`	`clubs`
תפקיד, סטטוס (`pending/active/suspended/banned/left/rejected`), תאריכים, `via`, נקודות דמו, העדפות הכרזות	`Membership[]` (clubId, userId)	`memberships`
מעקב אחרי מועדון בלי חברות (רואה רק טורנירים ציבוריים)	`Follow[]`	`follows`
הגדרה מלאה + סטטוס + `registrations[]`; `game` (וריאנט), `structure` (סטאק, רמות, גודל שולחן), `access` + `accessCode`, `tableBranding`, `scheduleId`, דגלי דמו (`simulateRegistrations`, `demoSpeed`, `seeded`)	`Tournament`	`tournaments`
סיכום טורניר שהסתיים: `places[]` (מקום, שחקן, נקודות, `bustedBy`), `entrants`, `game`, `seeded`, `invalidated`. **זה המקור היחיד לכל דירוג, ליגה, הישג ואנליטיקה**	`TournamentResult`	`results`
היסטוריית ידיים לשחקן, שמורה **כפי שהוא היה רשאי לראות** (קלפי יריבים רק אם נחשפו)	`HandRecord[]`	`hands`
עונות עם חוקי ניקוד מגורסאים וטבלה קפואה בסיום; הישגים עם מקור	`League`, `AchievementAward`	`leagues`, `achievements`

מה זה מחזיק	טיפוס	אוסף ב-DB
טיוטות שמורות ולוחות חוזרים (ימים, שעה מקומית, חלון פרסום)	`TournamentTemplate`, `RecurringSchedule`	`templates`, `schedules`
תזכורת אישית לטורניר (60/1440 דק׳ לפני), נשלחת פעם אחת	`PersonalReminder[]`	`reminders`
הכרזות מועדון עם קהל יעד (סגמנט/חברים/עוקבים), תזמון, ורשומת מסירה שמפרידה זכאים מנמסרים	`Announcement`, `DeliveryBatch`	`announcements`, `batches`
תיבת ההתראות של כל משתמש: `kind`, `key` (מפתח תרגום) + `params`, קישור לראוט, `dedupeKey`. נכתבת רק דרך `notify()` שמכבד העדפות ומדלג על בוטים	`Notification[]`	`notifications`
הודעות לערוצים חיצוניים (אימייל/SMS/וואטסאפ/פוש) — באב-טיפוס תמיד `status: 'simulated'`	`OutboxMessage[]`	`outbox`
צ׳אט שחקן↔מועדון, חסימות, דיווחים		`conversations`, `messages`, `blocks`, `reports`
פרטי למועדון: `profiles` (מפתח `clubId:userId`), `events` (ציר זמן, append-only), `notes`, `tags` (עם היסטוריית עריכה), `segments` (כללים או סטטי), `rules` + `runs` (אוטומציות), `links` (קישורי הזמנה כולל אישיים), `settings`, `imports` (רשומות הגירה)	`CrmStore`	`crm`
מקרי יושרה שנגזרו מההיסטוריה, עם סטטוס והערות	`IntegrityCase[]`	`integrity`
תמיכה, תוכניות SaaS, חשבוניות דמו, בריאות דמו, מונים		`tickets`, `plans`, `invoices`, `health`, `counters`
כל פעולת צוות משמעותית (`action` + פרמטרים), append-only	`AuditEntry[]`	`audit`
ניסיונות קוד גישה להגבלת קצב		`accessAttempts`

מכונת המצבים של טורניר (`TournamentStatus`): `DRAFT → SCHEDULED → REGISTRATION_OPEN → REGISTRATION_CLOSED → SEATING → RUNNING ⇄ PAUSED → COMPLETED`, ו-`CANCELLED` מכל מצב לפני הריצה. `RegistrationStatus`: `requested → approved → registered`, או `waitlisted`, `rejected`, `cancelled`. גישה (`access`): `members` (כל חבר פעיל), `approval` (חבר מבקש וצוות מאשר), `invite` (קוד גישה) — ובכל מקרה **קוד גישה לבדו לעולם לא עוקף את דרישת החברות**.

גרסאות סכימה: `SCHEMA_VERSION = 7` (ב-`seed.ts`). אין מיגרציות באב-טיפוס: שינוי גרסה מאפס את הדמו. בייצור כל אוסף הופך לטבלה (ראו `PRODUCTION_ARCHITECTURE.md` §4).

5. זהויות, תפקידים והרשאות

הרשאות הן תמיד **לפי מועדון**: אותו משתמש יכול להיות בעלים במועדון א׳ ושחקן רגיל במועדון ב׳. השער היחיד הוא `permissions.ts`: `can(db, userId, permission, clubId)` ו-`requirePermission(...)` (זורק `ForbiddenError`). אדמין פלטפורמה (`user.platformRole === 'platform_admin'`) נבדק בנפרד דרך `requirePlatform` ואינו חבר אוטומטי בשום מועדון. דירוג תפקידים (`ROLE_RANK`) מונע מחבר צוות לקדם מישהו מעל עצמו.

מה היא פותחת	support_agent	tournament_manager	manager	owner	הרשאה
כניסה לקונסולה	✓	✓	✓	✓	`console.access`
פרופיל ומיתוג			✓	✓	`club.edit_profile`, `club.branding`
מסד שחקנים, הפניות, התראות שימור	✓	✓	✓	✓	`members.view`
בקשות, השעיה, תפקידים			✓	✓	`members.approve`, `members.suspend`, `members.assign_roles`
אשף, שליטה חיה, תבניות, לוחות חוזרים, מצב סטרימר (`run`)		✓	✓	✓	`tournaments.create/edit/run/cancel/registrations`
הכרזות וקמפיינים		✓	✓	✓	`announcements.publish`
אנליטיקה, מפת חום, דוח שבועי		✓	✓	✓	`analytics.view`
יומן ביקורת, משחק הוגן			✓	✓	`audit.view`
פניות תמיכה		✓	✓	✓	`support.manage`
מנוי (דמו)				✓	`billing.view`
הערות ותיבה	✓		✓	✓	`crm.notes`, `crm.messages`
תגיות/סגמנטים, הגדרות, הגירה, ייצוא			✓	✓	`crm.edit`, `crm.configure`, `crm.export`

שחקן (`player`) אין לו הרשאות צוות בכלל; מה שהוא רואה נקבע ב-`selectors.ts` (`tournamentsForUser`, `canViewTournament`, `canWatchTournament`): חבר פעיל רואה את כל טורנירי המועדון, עוקב רואה רק `visibility: 'public'`, זר לא רואה כלום.

החלפת זהות בדמו (`session.ts` → `DemoIdentity`) משנה רק את `meId`; כל הנתונים משותפים באותו DB. בייצור `meId` מגיע מה-session המאומת והשאר לא משתנה.

6. מנוע הפוקר (`src/engine`)

המנוע הוא TypeScript טהור ללא DOM וללא תלויות, כך שהוא עובר לשרת כמו שהוא. כל יד היא רצף אירועים (`HandEvent[]`) שמוספים ל-`HandState`; כל מה שהממשק מציג נגזר מהאירועים האלה אחרי הסתרה.

נקודות מפתח	אחריות	קובץ
`deckFor('standard52' | 'short36')`, `shuffledDeck(rng, kind)`; `cryptoRng()` למשחק חי, `seededRng(seed)` לבדיקות ולדמו דטרמיניסטי	קלפים וחפירה	`cards.ts`, `rng.ts`
`VARIANTS: Record<VariantId, VariantRules>` — לכל משחק: משפחה, מספר קלפי יד, `holeUse: 'any' | 'exactly2'`, חפיסה, מבנה הימור (`no_limit` / `pot_limit` / `fixed_limit`), דירוג (`standard` / `short_deck`), `hiLo`, מינ'/מקס' שחקנים, `implemented`. מיושמים: nlhe, plhe, flhe, short_deck, plo4, plo5, plo6, plo8, flo, flo8. `RULE_MODIFIERS` (סטאד, דרו, run-it-twice ...) רשומים כ-planned וחסומים בממשק	רישום וריאנטים מוקלד	`variants.ts`
`startHand(table, blinds, rng, {variant})` → מחלק בליינדס/אנטי וקלפים; `legalActions(table)` → `{canFold, canCheck, canCall, callAmount, canBet, canRaise, minTo, maxTo, fixed}`; `applyAction(table, seat, action)`; רחובות `preflop → flop → turn → river → complete`; מקסימום פוט-לימיט = `min(stack, currentBet + pot + call)`; פיקסד-לימיט: הימור קטן/גדול, תקרת 4 הימורים (`FIXED_LIMIT_CAP`) כשיש יותר משני שחקנים חיים	מכונת מצבי היד (לכל הוריאנטים)	`holdem.ts`
`buildPots(contributions, deadMoney)` — חלוקה לפי רמות אול-אין, כל פוט עם רשימת הזכאים לו	פוטים צדדיים	`pots.ts`

נקודות מפתח	אחריות	קובץ
`evaluate(cards, ranking)` → ציון מספרי להשוואה; `bestHigh` מכבד `holeUse` (באומהה בדיוק 2 מהיד + 3 מהלוח); `evaluateLow8` + `bestLow` ללוא or-better-8; שורט דק: פלאש > פול האוס, A-6-7-8-9 הסטרייט הנמוך (מוסכמה מתועדת בראש `variants.ts`)	הערכת ידיים	`evaluator.ts`, `hands.ts`
`viewOf(table, viewerSeat)` בונה `TableView` בו קלפי יריבים הם `null` אלא אם נחשפו (`showdown` / `show`); `redactEvent` לשידור אירועים; `viewOfAll` חשוף לגמרי — **לשימוש צד-שרת בלבד** (מאגר הסטרימר המושהה)	**הסתרת קלפים**	`view.ts`
`submitCommand(table, {seat, handNo, seq, actionId, action})`: פקודה עם `actionId` שכבר הוחלה מחזירה `duplicate: true`; `seq` לא תואם → נדחה. זה החוזה לפקודות מעל WebSocket בעתיד	פקודות אידמפוטנטיות	`session.ts`
בוט מחליט לפי `preflopScore(hole, rules)` והערכת אקוויטי במונטה-קרלו (`estimateEquity`); כל בוט מסומן `isBot` ומוצג עם תג BOT	יריבים מדומים	`bots.ts`, `equity.ts`
פריסטים turbo/regular/deep/sng/custom, `presetStructure(id)`; רמות עם דקות ואנטי, `lateRegLevels`	מבנה בליינדס	`structure.ts`

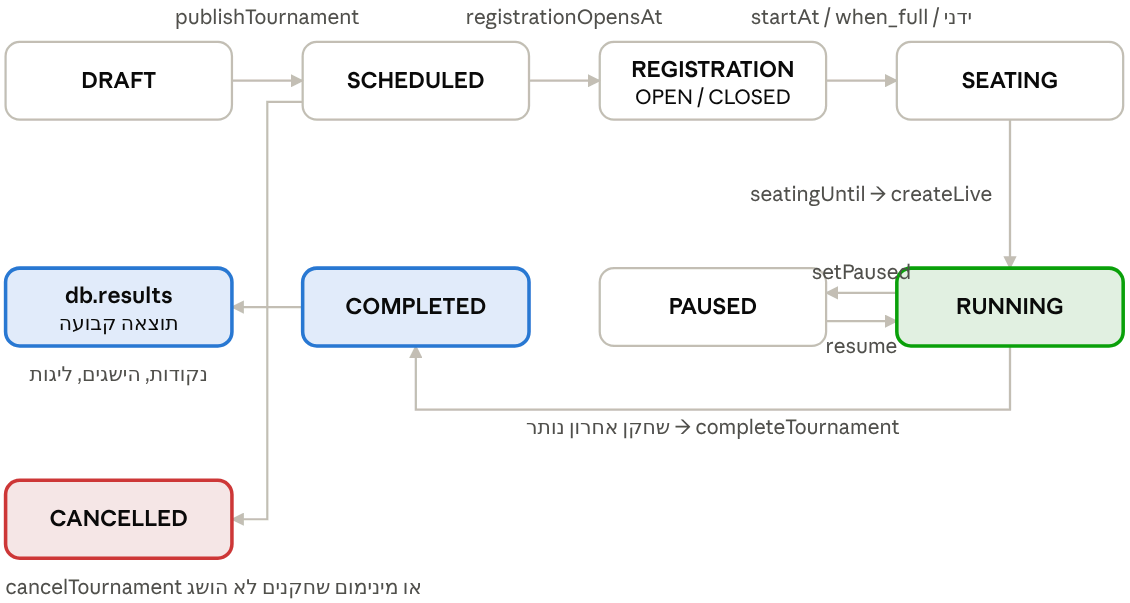
זרימת יד אחת: `start` (כפתור, בליינדס, סטאקים) → `post` → `hole` (לכל מושב; בהסתרה `cards: null` ליריבים) → `turn` / `action` לסירוגין → `collect` (בניית פוטים) → `street` (flop/turn/river) → ... → `showdown` / `show` → `award` (לפוט, עם `half: 'high' \| 'low'` בהי-לו, חלוקת צ׳יפ עודף למקום הקרוב לכפתור) → `end` (סטאקים, מודחים). התצוגה (`TableStage`) עובדת על `TableView` בלבד ולעולם לא על `HandState`.

בדיקות: `tests/engine.test.ts` (הולדם, פוטים, חפיסה ללא כפילויות, הסתרה, אידמפוטנטיות), `tests/variants.test.ts` (24 מקרים: אומהה בדיוק 2, פוט-לימיט, פיקסד-לימיט, שורט דק, הי-לו עם רבעים).

7. מנוע הטורניר וה-Runtime

טורניר חי הוא אובייקט `TournamentLive` (`engine/tournament.ts`) שחי בזיכרון ה-Runtime ונשמר ל-localStorage כל 3 שניות: שעון טורניר (`clockMs` , לא מתקדם בהשהיה), רמה נוכחית, שולחנות (`TTable[]` , כל אחד עם `TableState` של המנוע), שחקנים, הדחות, ציר זמן.

טורניר עובר מטיוטה לתוצאות; הטיק מקדם את המעברים המתוזמנים



מצבי טורניר · 8 מצבים, מעברים מתוזמנים וידניים

הטיק (`Runtime.tick` , כל 250ms) מקדם את המעברים המתוזמנים (פתיחת הרשמה, התחלה ב- `startAt` , סיום הושבה) והפעולות הידניות מגיעות מ- `api.ts` (`publishTournament` , `beginSeating` , `setPaused` , `cancelTournament`).

מה קורה בכל טיק (`runtime.ts`):

1. לכל טורניר לפני ריצה: הרשמות מדומות (אם `simulateRegistrations`), מעבר ל-SEATING בזמן ההתחלה או ביטול אם אין מינימום, ואז `createLive` ומעבר ל-RUNNING.
2. לכל טורניר רץ: `advanceClock` (כפול `speed` הדמו), `playDueHands` לשולחנות ללא אדם (יד שלמה מחושבת מיידית לפי קצב זמן משוער), `rebalance` (איזון שולחנות, שולחן סופי), הדחות ודירוג סופי.
3. שולחן שבו יושב אדם מנוהל על ידי `TableDriver` : הוא מציג אירועים בקצב אנושי, מפעיל בוטים עם עיכוב, מנהל טיימר תור ומקבל פעולות שחקן דרך `submitCommand` (עם `actionId`). יד שהסתיימה נרשמת ל- `db.hands` בגרסה המוסתרת של השחקן.
4. `captureFeeds` : צילום מצב כל שולחן רץ למאגר הצופים המושהה (ראו §13).

5. סיום: כשנשאר שחקן אחד → `api.completeTournament` כותב `TournamentResult`, מחלק נקודות (`rankingPoints`), מפעיל `evaluateAchievements`, מעדכן ליגות, שולח התראות תוצאה.
6. תחזוקה אחת לשנייה: הכרזות מתוזמנות, `automation.tick` (תזכורות מועדון ואישיות, כלל חוסר פעילות, התראות צוות, לוחות חוזרים), סימולציית צוות לדמו.

הושבה ואיזון: `createLive` מפזר נרשמים אקראית לשולחנות לפי `tableSize` (מוגבל לפי הוריאנט: PLO6 עד 6). `rebalance` מעביר שחקנים בין ידיים כשהפרש עולה על 1 וסוגר שולחנות כשאפשר. רישום מאוחר (`addLateEntrant`) מותר עד `lateRegLevels`.

המגבלה המהותית: הטיק רץ רק כל עוד הדף פתוח. סגירת הדפדפן מקפיאה את הטורניר (הוא ממשיך מהנקודה השמורה בפתיחה הבאה). בייצור ה-Runtime הוא תהליך שרת (ראו §17).

8. אפליקציית השחקן (`src/player`, `src/table`)

ניווט הוא דרך `go(route, params)` (`hooks.ts`); הראוט נשמר ב-session כך שרענון מחזיר לאותו מסך. אין URL routing (קובץ יחיד); קישורי הזמנה הם `https://join.pokerclubos.example/<token>` ומפוענחים ב-`api.openInvite`. המעטפת `PlayerShell` (v3) נותנת סרגל עליון בדסקטופ וטאבים תחתונים במובייל.

מה המסך עושה	קובץ	ראוט
בחירת זהות דמו ושפה	`Welcome.tsx`	`welcome`
פיד אישי: הטורניר הבא שלי, חי עכשיו, המלצות ממועדונים שלי, הכרזות	`Home.v3.tsx`	`home`
כל הטורנירים שאני רשאי לראות (`tournamentsForUser`), סינון לפי מועדון/משחק/מצב	`Tournaments.tsx`	`tournaments`
לובי: הרשמה/בקשה/קוד גישה, מבנה בליינדס, נרשמים, שולחנות חיים, דירוג חי, כפתור "צפייה בהשהיה"	`Lobby.tsx` + `lobby-hero.v3.tsx`	`tournament`
השולחן עצמו: מציג `TableView` מה-`TableDriver`, פעולות (fold/check/call/bet/raise), סליידר, פריסטים, טיימר), צלילים, ערכות נושא, לוגו מועדון/טורניר על הלבד, תג BOT, תזכורת זמן משחק	`table/TableScreen.tsx` + `TableStage.v3.tsx` + `ActionBar.tsx`	`table`

מה המסך עושה	קובץ	ראוט
צפייה בהשהיה / מצב סטרימר (§13)	`table/SpectatorScreen.tsx`	`spectate`
תוצאות טורניר שהסתיים: מקומות, נקודות, מי הדיח את מי	`Results.tsx`	`results`
המועדונים שלי + הצטרפות לפי קוד/קישור; דף מועדון עם מעקב, בקשת חברות, הודעה למארגן, דירוג מועדון, תפריט ⋯: הזמנת חבר (קישור אישי + QR), QR של המועדון, העדפות חברות, עזיבה	`Clubs.tsx` , `ClubPage.tsx`	`clubs` , `club`
פתיחת מועדון ב-4 צעדים (בסיס → מראה → טורניר ראשון → סיום עם QR) → `api.createClub`	`ClubWizard.tsx`	`club.new`
כל המועדונים שאני בצוות שלהם: חי, בקשות, תיבה, מתרחקים, קפיצה לקונסולה	`StaffOverview.tsx`	`staff`
דירוג גלובלי (קטגוריות, עונה/חודש, סינון משחק, כולל דמו), דירוגי מועדונים, ליגות; דף ליגה עם טבלה/טורנירים/חוקים	`Rankings.tsx` , `LeaguePage.tsx`	`rankings` , `league`
6 לשוניות: סקירה, סטטיסטיקות, היסטוריית טורנירים, דירוגים, הישגים, עונות; מתג פרופיל ציבורי	`Profile.tsx`	`profile`
חודש/שבוע/רשימה, תזכורות, ייצוא ics.	`Calendar.tsx`	`calendar`
היסטוריית ידיים + נגן (צעדים, רחובות, מהירות, מקלדת, העתקה כטקסט)	`Hands.tsx`	`hands`
צ׳אט עם מועדונים (לא בין שחקנים), חסימה, דיווח	`Messages.tsx`	`messages`
תיבת התראות עם קישורים לראוטים	`Notifications.tsx`	`notifications`
שפה, ערכת קלפים/שולחן, התראות לפי סוג, ערוצים (מדומים), משחק אחראי, פרטיות	`Settings.tsx`	`settings`

איך שחקן מגיע לשולחן: לובי → `api.registerForTournament` (בודק חברות פעילה, הפסקה עצמית, מקום פנוי → אחרת `waitlisted`; ב-`access: 'approval'` נוצר `requested` עד שצוות מאשר; ב-`invite` נדרש קוד בנוסף לחברות) → כשהטורניר עובר ל-RUNNING הלובי מציג "לשולחן שלי" → `TableScreen` נצמד ל-`runtime.attach(tournamentId, tableNo, seat)` ומקבל `TableView` מוסתר למושב שלו.

קומפוננטות משותפות (`src/ui` , `src/shared`): `core.tsx` (Button, Badge, Panel,) `forms.tsx` (PageHeader, EmptyState, Notice, DemoTag, BotTag,) `overlay.tsx` (Segmented, Checkbox, Switch, Slider,) `identity.tsx` (Dialog, Menu, Tabs, ConfirmDialog) `charts.tsx` (Avatar, ClubLogo, ClubCover, accentStyle), `shared/tournament.v3.tsx` (כרטיסי טורניר), `shared/competition.tsx` (טבלאות דירוג), `shared/qr.tsx` (Field, TextInput, Select,).

9. קונסולת המועדון (`src/console`)

נכנסים אליה רק עם הרשאת `console.access` במועדון (`clubId` חובה בראוט). ה-`ConsoleShell` (v3) מציג sidebar עם קבוצות: סקירה · טורנירים · ליגות · CRM · תקשורת · הגדרות; ובמובייל — drawer. כל פעולה בקונסולה נרשמת ב-`db.audit` דרך `audit(db, actorId, clubId, action, meta)`.

ראוט	קובץ	מה עושה	פונקציות עיקריות
`console.home`	`Dashboard.tsx`	של המועדון: חברים, בקשות ממתינות, טורניר הבא, חי עכשיו, התראות שימור, פעילות אחרונה	`selectors.clubPulse`, `retention.alerts`
`console.tournaments`	`Tournaments.tsx`	רשימת טורנירים לפי סטטוס, יצירה (ריק / מתבנית / מומלץ), שכפול, ביטול	`api.listClubTournaments`, `scheduling.templates`
`console.wizard`	`wizard/Wizard.tsx` + `steps/*`	אשף בן 6 צעדים — ראו להלן	`api.saveTournamentDraft`, `api.publishTournament`
`console.tournament`	`TournamentControl.tsx`	שליטה חיה: שעון/רמה, Pause/Resume, אישור בקשות, הושבה ידנית, הוספת late entrant, שולחנות חיים עם redaction, סיום/ביטול	`api.approveRegistration`, `api.openSeating`, `api.pauseTournament`, `api.completeTournament`, `runtime.*`
`console.leagues`, `console.league`	`Leagues.tsx`, `LeagueEditor.tsx`	ליגות/עונות: יצירה, preset ניקוד, צירוף טורנירים, נעילה וסיום (§11)	`competition.createLeague`, `attachTournament`, `completeLeague`
`console.crm.*`	`crm/*`	מודול CRM מלא (§10)	`crm.*`, `analytics.*`, `automation.*`
`console.announcements`	`Announcements.tsx`	הודעות למועדון: טיוטה → תצוגת קהל (`audiencePreview`) → פרסום → batch משלוח	`api.publishAnnouncement`, `channels.dispatch`

פונקציות עיקריות	מה עושה	קובץ	ראוט
`api.updateBranding`, `api.saveTablePreset`	לוגו, cover, צבע מבטא, ערכות שולחן (`THEMES`), presets ברירת מחדל לטורנירים, תצוגה מקדימה חיה של שולחן	`Branding.tsx`	`console.branding`
`api.setRole`	צוות: הוספה/שינוי/הסרה של תפקיד, אסור להוריד את הבעלים האחרון	`Roles.tsx`	`console.roles`
`db.audit` (קריאה דרך selector)	יומן ביקורת מסונן לפי פעולה/משתמש/תאריך	`Audit.tsx`	`console.audit`
`integrity.*`	תור יושרה של המועדון (§13)	`Integrity.tsx`	`console.integrity`
`api.openTicket`, `db.plans`	כרטיסי תמיכה לפלטפורמה; תוכנית המנוי (דמו, ללא תשלום)	`Support.tsx`, `Subscription.tsx`	`console.support`, `console.subscription`

אשף הטורניר (`console.wizard`) — טיוטה נשמרת אחרי כל צעד (`status: DRAFT`), ולכן אפשר לצאת ולחזור:

1. **משחק** (`GamePicker`): כרטיסי וריאנט מ-`VARIANTS`; פורמטים שאינם זמינים מוצגים כ-"מתוכנן" ולא ניתנים לבחירה. גודל שולחן מוגבל לפי `maxSeats` של הוריאנט (PLO6 עד 6).
2. **מבנה**: preset (turbo/regular/deep/sng/custom), עריכת רמות, הפסקות, late registration, re-entry, סטארטינג סטאק.
3. **גישה ורישום**: `visibility`, `access` (invite / approval / members + קוד), מקסימום שחקנים, רשימת המתנה, חלונות רישום.
4. **שולחן ומיתוג**: ירושה מהמועדון או override לטורניר (ערכה, לוגו, צבע). מה שנשמר הוא `tableBranding`; ב-Tournament `resolveTableBranding(db, t)` מחשב את התוצאה הסופית (טורניר → מועדון → פלטפורמה).
5. **תזמון**: תאריך/שעה באזור הזמן של המועדון, כפתור "השתמש בזמן המומלץ" מתוך `retention.recommendations`, תזכורות (60/1440 דק'), אפשרות להפוך ל-recurring או לשמור כתבנית.
6. **סיכום ופרסום**: ולידציה (`validateDraft`) מחזירה שגיאות בקודים כמו `game_not_available`, `too_many_seats`, `start_in_past` (→ `publishTournament`) מעביר ל-SCHEDULED/REGISTRATION_OPEN ומפיץ `new_tournament` לפי כללי האוטומציה.

שליטה חיה (`TournamentControl`): כל שולחן מוצג דרך `runtime.publicView(tournamentId, tableNo)` — כלומר גם לצוות **לא** נחשפים קלפים מוסתרים; רק מצב הסטרימר (§13) רואה אותם, ובהשהיה. פעולות הצוות עוברות ולידציה בשכבת השירות (למשל `pauseTournament` רק ב-RUNNING, `completeTournament` רק כשנותר שחקן אחד או ידנית עם אישור).

10. מודול ה-CRM (`src/platform/crm.ts`, `analytics.ts`, `automation.ts`, `src/console/crm/*`)

מודל הנתונים. `db.crm` מכיל: `profiles` (מפתח `clubId:userId` — פרופיל נפרד לכל מועדון, עם `tags`, `assignedStaff`, `source`, `referrerId`, `firstSeen`, `lastActive`, `consent`), `events` (ציר זמן append-only; כל אירוע `{id, clubId, userId, type, at, meta}`), `notes`, `segments`, `links` (קישורי הזמנה: `token`, `clubId`, `createdBy`, `kind: 'club'|'personal'`, `campaign`, `opens`), `settings` (לכל מועדון), `imports`. כל קריאה/כתיבה עוברת רק דרך `crm.*` — מסכים אינם רשאים לקרוא `db.crm.*` ישירות (בדיקת ארכיטקטורה ב-`tests/crm.test.ts`).

סוגי אירועים (`CrmEventType`): `link_opened`, `followed`, `unfollowed`, `membership_requested/cancelled/approved/rejected/suspended/reinstated/banned/unbanned/left`, `role_changed`, `tag_added/removed`, `staff_assigned`, `note_added`, `tournament_requested/registered/approved/rejected/revoked/waitlisted/unregistered`, `message_sent/received`. האירועים נכתבים על-ידי `api.ts` ו-`messaging.ts` דרך `crm.track(db, clubId, userId, type, meta)`, כך שה-CRM הוא תוצר לוואי של כל פעולה במערכת ולא מודול שצריך לעדכן ידנית.

מה עושה	קובץ	מסך
כרטיסי מצב: חברים פעילים, בקשות פתוחות, התראות שימור, סגמנטים מובילים, פעילות אחרונה	`crm/Overview.tsx`	`console.crm`
טבלת שחקנים עם `crm.queryPlayers(db, actor, clubId, {segmentId, tag, search, sort, page})`; פעולות מרובות: תיוג, שיוך איש צוות, הודעה	`crm/Players.tsx`	`console.crm.players`
פרופיל 360°: סטטוס חברות, מקור/מפנה, "איך הם משחקים" (`retention.playerPreferences`), ציר זמן אירועים, הערות, תגיות, טורנירים, הודעות; פעולות: אישור/השעיה/חסימה, forget (מחיקת נתונים אישיים לפי `crm.forgetPlayer`)	`crm/PlayerProfile.tsx`	`console.crm.player`
תור בקשות חברות + בקשות לטורנירים באישור; מציג מפנה אם יש; אישור/דחייה עם סיבה	`crm/Requests.tsx`	`console.crm.requests`
סגמנטי מערכת (קבועים) + סגמנטים מותאמים; בונה חוקים (ראו להלן); ספירה חיה; שימוש כקהל להודעות/אוטומציה	`crm/Segments.tsx`	`console.crm.segments`
שיחות מועדון↔שחקן (`messaging.conversations`), תגובה, סימון, חסימה; דיווחים	`crm/Inbox.tsx`	`console.crm.inbox`
הודעות/קמפיינים: טיוטה → קהל (סגמנט/כולם/רשימה) → תצוגה מקדימה של גודל קהל אחרי העדפות והסכמות → שליחה → `batches` עם ספירה לפי ערוץ (`in_app` אמיתי; השאר `simulated`)	`crm/Notifications.tsx`	`console.crm.notifications`

מה עושה	קובץ	מסך
לשוניות: סקירה · משפך · KPI · שימור (cohorts) · רכישה (מקורות/הפניות) · מעורבות · **זמנים מומלצים** (heatmap יום×שעה) · **דו"ח שבועי** (העתקה כטקסט / שליחה לצוות)	`crm/Analytics.tsx`	`console.crm.analytics`
חוקים: `welcome`, `new_tournament`, `reminder`, `membership_request`, `inactive`; הפעלה/כיבוי, עריכת טקסט, ערוץ; יומן ריצות	`crm/Automation.tsx`	`console.crm.automation`
דשבורד הפניות: מפנים מובילים, משפך opened→requested→approved→played→returning, קמפיינים, drill-down לשחקנים	`crm/Referrals.tsx`	`console.crm.referrals`
אשף ייבוא CSV: מיפוי עמודות → תצוגה מקדימה → רשומות ממתינות (לא נוצרים חשבונות!) → צירוף אוטומטי לפרופיל כשהשחקן האמיתי מאושר (לפי טלפון/אימייל)	`crm/Migration.tsx`	`console.crm.import`
שמירת נתונים (retention בימים → `applyRetention`), הסכמות ברירת מחדל, opt-out מועדון מערוצים, ייצוא CSV/JSON	`crm/Settings.tsx`	`console.crm.settings`

סגמנטים. סגמנט = `{id, clubId, name, rules: Rule[], system?: boolean}`; `Rule = {field, op, value}`. שדות נתמכים: `status`, `tag`, `lastActiveDays`, `tournamentsPlayed`, `joinedDays`, `source`, `referredBy`, `game` (nlhe/omaha...), `leagueMember`, `vip`, `cashesCount`. סגמנטי מערכת (`SYSTEM_SEGMENTS`) מחושבים ולא ניתנים לעריכה: חדשים (7 ימים), פעילים, לא פעילים 7/14/30, VIP, שחקני Hold'em, שחקני Omaha, משתתפי ליגה, מופנים. `crm.evaluateSegment(db, clubId, segment)` מחזיר `userId[]`; הקהלים של הודעות ואוטומציה מתבססים עליו.

אוטומציה (`automation.ts`): `tick(db, now)` רץ מתוך `Runtime.tick` כל שנייה, מחשב לכל חוק פעיל את היעדים ומפעיל `core.notify` עם מפתח dedupe (`rule:userId:anchor`) — לכן ריצה חוזרת לעולם לא שולחת כפול. החוק `inactive` נשען על `retention.alerts` (סעיף 13).

הודעות וערוצים. `core.notify(db, {userId, clubId, kind, title, body, link, channels?})` → בודק העדפות המשתמש לפי סוג (`settings.notifications[kind]`), מדלג על בוטים, מבצע dedupe → `fanOut` כותב `db.notifications` (in-app, אמיתי) ומוסיף שורות `outbox` לערוצים חיצוניים → `channels.dispatch` מעביר לאדפטרים. ללא אדפטר אמיתי השורה נשארת `status: 'simulated'` ו**לעולם לא נספרת כנמסרה** בסטטיסטיקות.

אנליטיקס (`analytics.ts`): `compute(db, clubId)` בונה אינדקס אחד של participations (`{userId, tournamentId, game, place, field, at}`) וממנו נגזרים כל הדו"חות: `overview`, `funnel` (פתיחת קישור → בקשה → אישור → טורניר ראשון → חוזר), `kpi` (פעילים/חדשים/נטישה לפי תקופה), `retention` (cohorts חודשיים), `acquisition` (לפי `source`/מפנה/קמפיין), `engagement` (תדירות, משחק מועדף). כל הערכים מחושבים מהנתונים ב-DB — אין מספרים מומצאים; במצב דמו הדאטה עצמה מסומנת.

11. שכבת התחרות (`src/platform/competition.ts`)

ציון ביצוע לטורניר בודד (`performanceScore(place, field, opts)`):

```
base  = 100 × (1 − (place − 1) / field)
bonus = 25 אם place = 1, אחרת 10 אם שולחן סופי (place ⩽ 9 וגם ⩽ field/4), אחרת
0
size  = min(2, 0.5 + log10(field))          // שדה גדול שווה יותר, עד 2×
sng   = 0.8 אם הפורמט sit&go, אחרת 1
score = round((base + bonus) × size × sng)
```

לוחות דירוג (`globalLeaderboard` , `clubLeaderboard`): קלט = `db.results` (רק `COMPLETED` , לא `invalidated`), חלון זמן (all-time / עונה / חודש), סינון משחק, קטגוריות (נקודות, ניצחונות, שולחנות סופיים, ממוצע ביצוע, ITM%). **כללי הוגנות** (מוצאים מהדירוג): בוטים; תוצאות דמו מה-seed אלא אם השחקן הפעיל `includeDemo` ; טורנירים מבוטלים; תוצאות שסומנו `invalidated` ע"י יושרה; פרופילים פרטיים (`settings.profileVisibility = 'private'`) יורדים מהלוח הגלובלי. סף כניסה: `MIN_TOURNAMENTS = 3` לדירוג הגלובלי.

ליגות ועונות (`db.leagues`): `League = {id, clubId, name, season, status: 'draft'|'active'|'completed', scoring: ScoringRules, scoringVersion, tournamentIds, finalStandings?}` . שלושה presets לניקוד: `position` (טבלת נקודות לפי מקום), `performance` (הנוסחה לעיל), `wins_and_tables` (נקודות על ניצחון/שולחן סופי/השתתפות) + `custom` . שינוי חוקי ניקוד בליגה פעילה מעלה `scoringVersion` ונרשם ב-audit; הטבלה מחושבת תמיד מהתוצאות לפי החוקים הנוכחיים (`leagueStandings`). `completeLeague` מקפיא `finalStandings` — מרגע זה ההיסטוריה אינה משתנה גם אם תוצאה תיפסל בהמשך. הצטרפות לליגה אוטומטית עם ההשתתפות הראשונה בטורניר מצורף.

דרגות (`TIERS`): Bronze → Silver → Gold → Platinum → Diamond לפי נקודות מצטברות בחלון מתגלגל של 90 יום; סמלים מקוריים (SVG ב- `shared/competition.tsx`). הדרגה מוצגת ליד השם בפרופיל, בלובי ובשולחן.

הישגים (`achievements.ts` בתוך competition): מנוע הצהרתי — רשימת `ACHIEVEMENTS` עם `check(ctx) => boolean` ; `evaluateAchievements(db, userId)` רץ אחרי כל `settleHand` / `completeTournament` /אישור הפניה ופותח רשומות `db.achievements` (`userId, key, at`) עם התראה. דוגמאות: `first_tournament` , `first_win` , `final_table_5` , `omaha_win` , `referral_first` , `referral_five` , `league_champion` . הוספת הישג = רשומה במערך + מפתח i18n.

12. תזמון ויומן (`scheduling.ts` , `calendar.ts`)

תבניות (`db.templates`): שמירה מתוך האשף (`saveTemplate(db, actor, clubId, draft, name)`) — נשמר הכל חוץ מתאריך/שעה/שם; `fromTemplate` מחזיר טיוטה חדשה עם השדות האלה ריקים.

תזמון חוזר (`db.schedules`): `RecurringSchedule = {id, clubId, templateId, weekday, time: 'HH:mm', timezone, leadDays, status: 'active'|'paused', lastRunAt}` . חישוב מופעים ב- `occurrences(schedule, from, to)` + `zonedTime(date, tz)` — השעה מוגדרת **בזמן המועדון**, לכן מעבר שעון קיץ אינו מזיז את "יום שלישי 20:00" (בדיקה חוצת-DST ב- `scheduling.test.ts`). היישום משתמש ב- `Intl.DateTimeFormat` בלבד, ללא ספרייה. `runSchedules(db, now)` רץ מה-tick: לכל תזמון פעיל מחשב מופעים בחלון `[now, now + leadDays]` , ויוצר טורניר מפורסם לכל מופע ש**אין** עדיין טורניר עם `{scheduleId, startAt}` זהה — אידמפוטנטי, אפשר להריץ אינסוף פעמים. טורניר שנוצר מתזמון נושא תג "נוצר אוטומטית" בקונסולה. השהייה (`pause`) מפסיקה יצירה חדשה אבל לא נוגעת בטורנירים שכבר פורסמו; מחיקה אותו דבר.

יומן אישי (`calendar.eventsFor(db, userId, from, to)`): מאחד (1) טורנירים שאני רשום אליהם, (2) טורנירים פתוחים במועדונים שאני חבר/עוקב בהם, (3) תאריכי ליגה (פתיחה/סיום עונה), (4) מופעים עתידיים מתזמונים חוזרים שטרם פורסמו (מסומנים "מתוכנן"). כל אירוע `{id, kind, at, title, clubId, tournamentId?, mine}` . תצוגות חודש/שבוע/רשימה הן UI בלבד על אותו מערך.

תזכורות אישיות (`db.reminders`): `{userId, tournamentId, minutesBefore: 60|1440}` ; `calendar.tickReminders(db, now)` מפעיל `core.notify` כשהזמן מגיע ומסמן `firedAt` (ללא כפילות). נפרד מתזכורות המועדון (חוק אוטומציה `reminder`).

ייצוא .ics (`calendar.toIcs(events, tz)`): מייצר VCALENDAR עם VEVENT לכל אירוע (UID יציב = מזהה האירוע, DTSTART ב-UTC, URL הזמנה אם קיים); ההורדה נעשית בדפדפן דרך Blob.

13. צפייה, יושרה, ערוצים, משחק אחראי, נגן ידיים

צפייה בהשהיה (`runtime.ts` → `feeds` , `spectate`). לכל שולחן חי ה-Runtime שומר ring buffer של פריימים `{at, publicView, fullView}` (עד 10 דקות / 1,200 פריימים).
`spectate(tournamentId, tableNo, mode)` מחזיר את הפריים **החדש ביותר שגילו ≤ השהייה**: מצב `public` — `SPECTATE_MIN = 30s` , מחזיר `publicView` (בלי קלפים מוסתרים, זמין לכל מי שרואה את הטורניר); מצב `streamer` — `STREAM_MIN = 60s` , מחזיר `fullView` (כל הקלפים) ודורש `tournaments.manage` במועדון. פריים צעיר מההשהייה לעולם לא מוחזר — בדקות הראשונות המסך מציג "ממלא מאגר..." עם ספירה לאחור. `SpectatorScreen` מציג את הפריים עם השהייה מצוינת, ובמצב סטרימר — רקע ירוק (chroma) ופריסה ל-overlay. בפרודקשן המאגר עובר לשרת השולחנות והלקוח נרשם לערוץ `table:{id}:delayed` .

יושרה (`integrity.ts`, מסך `console.integrity` + `platform.integrity`). `scan(db, clubId?)` עובר על היסטוריית הידיים והתוצאות ומייצר אותות: `bust_pattern` (שחקן מודח שוב ושוב מול אותו יריב), `co_finish` (זוג שמסיים יחד בשולחן סופי בתדירות חריגה), `win_rate` (אחוז ניצחון מעל סף לאורך מספר טורנירים). ספים ב- `THRESHOLDS` בראש הקובץ. כל אות נפתח כ- `IntegrityCase {id, clubId, userIds, signal, evidence, status: 'new'|'reviewing'|'cleared'|'escalated', notes[]}`; פעולות צוות: סימון בבדיקה, ניקוי, הסלמה לפלטפורמה, פסילת תוצאה (`invalidateResult` → יורד מהדירוגים ומהליגות). הכל נרשם ב-audit. **חשוב:** האותות מחושבות ממשחק מדומה (בוטים), ולכן המסך נושא באנר קבוע שאומר זאת. האלגוריתם עצמו אמיתי וירוץ על נתונים אמיתיים ללא שינוי.

ערוצים (`channels.ts`): רישום אדפטרים `registerAdapter(channel, {send(msg) => Promise<Result>})` ל- `in_app | email | sms | whatsapp | push`. בפרוטוטיפ רשום רק `in_app` אמיתי; לשאר יש אדפטר דמו שכותב `outbox` עם `status: 'simulated'`. `summary(db, clubId)` מחזיר ספירות `sent / simulated / failed` בנפרד — ה-UI מציג "מדומה" ולא "נמסר". הסודות של הספקים (Twilio, SES...) יירשמו בעתיד בצד השרת בלבד; אסור להכניס אותם לקוד הלקוח.

משחק אחראי (`settings.responsible`): `sessionReminderMin` — השולחן מציג תזכורת זמן כל X דקות משחק רצוף; `breakUntil` — הפסקה עצמית שהשחקן קובע לעצמו (`api.setBreak`); כל עוד `onBreak(db, userId)` אמת, `registerForTournament` נכשל עם `on_break` והלובי מסביר זאת. אין מגבלות הפקדה/הפסד כי אין כסף במוצר.

נגן ידיים (`player/Hands.tsx`): קורא `HandRecord {id, tournamentId, events: HandEvent[]}` מ- `db.hands` ומשחזר מצב שולחן צעד-צעד (reduce על האירועים עד האינדקס הנוכחי). קפיצה לרחוב (preflop/flop/turn/river/showdown), מהירות ×0.5/1/2, מקלדת (← → רווח Home End), "העתק כטקסט" מייצר היסטוריית יד טקסטואלית. הידיים נשמרות מנקודת המבט של השחקן (קלפי יריבים רק אם נחשפו ב-showdown).

14. קונסולת הפלטפורמה (`src/platform-admin`)

נגישה רק ל- `users[].platformAdmin === true` (בדמו: `u_admin`), נאכפת ב- `requirePlatform`. זו שכבת ה-SaaS מעל המועדונים.

אמיתי / דמו	מה מציג	ראוט
אמיתי (מחושב מה-DB)	סיכום רוחב-פלטפורמה: מועדונים, משתמשים, טורנירים חיים, כרטיסים פתוחים	`platform.home`

אמיתי / דמו	מה מציג	ראוט
אמיתי	רשימת tenants, סטטוס (active/suspended), תוכנית, בעלים; דף מועדון עם השעיה/החזרה, שינוי תוכנית, יצירת tenant חדש (`api.createTenant`)	`platform.clubs` , `platform.club`
אמיתי כקונפיגורציה; **אין חיוב**	קטלוג תוכניות (`db.plans` : שם, מגבלות חברים/טורנירים, פיצ׳רים)	`platform.plans`
דמו — נתונים מה-seed, ללא סליקה, מסומן ב-UI	חשבוניות למועדונים (`db.invoices`)	`platform.billing`
דמו — אין שרת למדוד; מסומן	בריאות מערכת: uptime, latency, שגיאות	`platform.health`
אמיתי	כל כרטיסי התמיכה מכל המועדונים (`db.tickets`), מענה, סגירה	`platform.support`
אלגוריתם אמיתי, נתונים מסימולציה (§13)	תור יושרה מאוחד — מקרים שהוסלמו מהמועדונים + סריקה כלל-פלטפורמית	`platform.integrity`

השעיית מועדון (`suspendClub`) חוסמת כניסה לקונסולה ורישום לטורנירים שלו, אבל לא מוחקת נתונים. מגבלות התוכנית נאכפות ב- `api.publishTournament` (`plan_limit`) כשעוברים את מכסת הטורנירים החודשית) וב- `approveMembership` .

15. תרגום, עיצוב v3, RTL ומובייל

i18n (`src/i18n/`): קבצי מפתחות לפי שפה ואזור: `en.ts` / `he.ts` (בסיס), `*.crm.ts` , `*.admin.ts` , `*.v3.ts` , `*.v4.ts` — ממוגים למילון אחד ב- `index.ts` . שימוש: `const t = useT(); t('lobby.register')` , עם פרמטרים `t('x.y', {name})` וריבוי `t.n('players', count)` (מפתחות `.one/.other`). `scripts/i18n-check.mjs` מוודא שלכל מפתח באנגלית יש השפה נשמרת. `npm test` מקביל בעברית ושאין מפתחות ללא שימוש — רץ במסגרת ב- `settings.locale` ומשפיעה גם על `dir` של ה- `<html>` .

RTL: כל הפריסות משתמשות במאפייני logical (`ms-` , `me-` , `ps-` , `text-start` ולא left/right; תמיד LTR השולחן עצמו (מיקום מושבים) הוא .(מראה אופקי בעברית) `flip-rtl` אייקוני כיוון מקבלים — מושב 1 תמיד באותו מקום פיזי בשתי השפות.

עיצוב "Graphite & Ember — v3" (`styles/app-v3.css`, נבחר עם `--design=v3`): טוקנים כמשתני CSS — `--bg`, `--surface`, `--surface-2`, `--line`, `--ink`, `--ink-2`, `--muted`, `--` (accent) `ember`, `--felt` (שולחן), `--good/--warn/--bad`; טיפוגרפיה אחת לכותרות וטקסט, מספרים ב-tabular-nums. מעטפות v3: `PlayerShell`, `ConsoleShell`, `AdminShell`, `TableStage.v3`, `lobby-hero.v3`, `Home.v3`; שאר המסכים משותפים לשתי הגרסאות ומקבלים את המראה מהטוקנים. העיצוב הקודם (v2) שמור ב-`styles/app.css` ובענף `backup/v0.2-original-design`; בנייה עם `--design=v2` מחזירה אותו במלואו.

מיתוג שולחן (`platform/table-branding.ts`): `THEMES` (ערכות קלף/לבד מקוריות), `resolveTableBranding` (טורניר → מועדון → פלטפורמה), לוגו מועדון על הלבד, צבע מבטא לכפתורים/צ'יפים.

רספונסיביות: breakpoints של Tailwind (`sm` 640, `md` 768, `lg` 1024, `xl` 1280). מתחת ל-`md` — ניווט תחתון בשחקן, drawer בקונסולה, טבלאות ב-`overflow-x-auto`, `Segmented` גולל אופקית, שולחן בפריסה אנכית עם סרגל פעולות מוצמד. סריקת QC אוטומטית (`node scripts/e2e.mjs --only=S`) מצלמת כל מסך ב- 1440 / 1280 / 768 / 390 / 360 ונכשלת על גלילה אופקית לא צפויה או טקסט חתוך.

16. בדיקות ו-QA

כל המספרים כאן הם מהריצה המתועדת ב-`docs/V04_REPORT.md` על הקומיט `adefd7c` (תג `v0.4.0`). לפני כל שינוי — להריץ מחדש ולא להסתמך על המספרים האלה.

תוצאה מתועדת	קבצים	כלי	סוג
90 / 90	`engine`, `variants`, `tournament`, `table-branding`, `i18n`, `qr`	`node --import tsx --test`	יחידה — מנוע
41 / 41	`competition`, `referral`, `calendar`, `retention`, `migration`, `scheduling`, `spectator`, `integrity`, `channels`	נ"ל	יחידה — מודולי v0.4
39 / 39	`platform` (runtime, scheduler, notify, permissions)	נ"ל	יחידה — פלטפורמה
50 / 50	`crm` (כולל בדיקות ארכיטקטורה)	נ"ל	יחידה — CRM

תוצאה מתועדת	קבצים	כלי	סוג
165 / 165	A (15) · שחקן B (19) · בעלים C · E (54) · CRM D (14) · פלטפורמה (28) · v0.4 מסעי F · כללי גישה (23) · מובייל/עברית (12) S smoke	Playwright, `scripts/e2e.mjs --only=X`	E2E
0 מסכים מסומנים	37 מסכים × 5 × EN/HE רוחבים	חלק מ-S	רספונסיבי QC
נקי	strict + `noUnusedLocals`	`tsc --noEmit`	טיפוסים
0 חסרים	כל המילונים	`scripts/i18n-check.mjs`	תרגום

איך מריצים. יחידה: `node --import tsx --test tests/engine.test.ts` … `tests/variants.test.ts` — להריץ בארבעה חלקים כמו בטבלה (הרצת הכל ביחד מגיעה ל-timeout בגלל CRM). E2E: קודם `node scripts/build.mjs --design=v3`, אחר כך `node scripts/e2e.mjs --only=A` (או B/C/D/E/F/S); **לא לבנות בזמן שה-e2e רץ** — הוא טוען את `dist/index.html` מהדיסק. המסעות מתחלים מזהות דמו ומבצעים פעולות אמיתיות ב-UI (כולל משחק ידיים שלמות מול בוטים במסע A).

מה הבדיקות מכסות בפועל (דוגמאות חשובות למתכנת):

- `engine.test.ts` / `variants.test.ts`: דירוג ידיים (כולל Omaha בדיוק 2 מהיד, Short Deck, Hi/Lo עם quartering), חלוקת קופות צדדיות ושבב בודד, חוקיות פעולות בכל רחוב, pot-limit max, fixed-limit cap, **redaction** — `viewOf` לעולם לא מחזיר קלפי יריב.
- `tournament.test.ts`: שעון, איזון שולחנות, late entry, תוצאות ונקודות, דטרמיניזם עם `seededRng`.
- `platform.test.ts`: הרשאות, קוד גישה לא עוקף חברות, השעיה חוסמת, התראות עם dedupe והעדפות, scheduler אידמפוטנטי, אין כסף בשום מקום.
- `crm.test.ts`: סגמנטים, אוטומציה, הסכמות, retention/forget, ייצוא; **בדיקת ארכיטקטורה** שסורקת את `src/player`, `src/console`, `src/platform-admin` ונכשלת אם מסך קורא `db.crm.*`, `accessCode` או `compute.` ישירות — זה מה ששומר על הגבול בין UI לשכבת השירות לקראת המעבר לשרת.
- `spectator.test.ts`: פריים צעיר מ 30s לא מוחזר; סטרימר דורש הרשאה; פריים ציבורי ללא קלפים מוסתרים.
- `scheduling.test.ts`: מופע שחוצה מעבר DST נשאר באותה שעה מקומית; הרצה כפולה לא מכפילה טורנירים.

- `channels.test.ts`: שורה `simulated` לעולם לא נספרת כ-sent; הפסקה עצמית חוסמת רישום.

מה לא מכוסה אוטומטית (מסומן UNVERIFIED ב-`V04_FEATURE_AUDIT.md`): רנדור "אלופים קודמים", לשוניות הפרופיל מעבר לטעינה, שליחת הדו"ח השבועי, פריסת overlay לסטרימר, נגן הידיים מעבר לצעד-צעד במסע A, תצוגת רב-מועדונים לצוות.

17. מה אמיתי, מה דמו, מה חסר — והדרך לייצור

זה הסעיף החשוב ביותר למתכנת. הפירוט המלא, פיצ'ר אחר פיצ'ר, נמצא ב-`docs/V04_FEATURE_AUDIT.md`.

מה נכלל	קטגוריה
מנוע הפוקר על כל הוריאנטים, מנוע הטורניר, הרשאות וכללי גישה, CRM כולל סגמנטים/אוטומציה/אנליטיקס, ליגות ודירוגים, הפניות ו-QR, יומן ותזכורות, תבניות ותזמון חוזר, צפייה בהשהיה, משחק אחראי, ייבוא חברים, יומן ביקורת, הודעות in-app, מיתוג שולחן	**אמיתי — לוגיקה מלאה שעוברת לשרת כמו שהיא**
יריבים (בוטים מסומנים), אותות יושרה (מחושבות ממשחק בוטים), ערוצים חיצוניים (email/SMS/WhatsApp/push → `simulated`), חיוב וחשבוניות, בריאות מערכת, כל נתוני ה-seed (מועדונים, שחקנים, היסטוריה)	**דמו / מדומה — הלוגיקה קיימת, הנתונים לא**
שרת, אימות (auth), מסד נתונים, WebSockets/מולטיפלייר אמיתי בין מכשירים, משחקי Stud/Draw/Mixed, פורמטי bounty/satellite/shootout, ייבוא היסטוריית טורנירים, תצוגות Open Graph לקישורים, דריסת מיתוג ברמת פלטפורמה, אפליקציות native, הישגי רצף ייעודיים	**חסר — לא קיים בקוד**
כל דבר כספי: הפקדות, משיכות, הימורים, תשלומים. המטבע היחיד הוא נקודות דמו	**מחוץ למוצר בכוונה**

הדרך לייצור — מפורטת ב-`docs/PRODUCTION_ARCHITECTURE.md`, והעיקרון הוא שכל שלב משאיר את הקודם עובד (ללא rewrite):

1. **חילוץ API** — לעטוף את `platform/*` בשרת (Fastify) Node עם `DB` בזיכרון מה-seed; לכתוב `src/platform/client.ts` שמשקף את אותן חתימות וקורא HTTP. המסכים מחליפים import בלבד. `actorId` מגיע מה-session, לעולם לא מה-body. הבדיקות הקיימות רצות כמו שהן מול שכבת השירות.
2. **התמדה** — PostgreSQL דרך שכבת repository; `types.ts` נשאר מקור האמת לסכמה (מיפוי אוסף→טבלה במסמך). מזהים נשארים מחרוזות עם קידומת (`t_`, `u_`) כך שה-fixtures מיובאים כמו שהם.
3. **העברת השולחנות** — `TournamentLive` רץ כ-actor לכל שולחן בשירות שולחנות; הלקוח מקבל `TableView` דרך WebSocket; `TableDriver` הופך ללקוח WS. החפיפה וה-RNG עוברים לשרת (`crypto.randomInt`); המאגר המושהה עובר איתו.
4. **אימות ו-tenancy** — magic link / OTP, עוגיות HttpOnly, row-level security לפי `club_id`. `permissions.ts` הופך ל-middleware כמו שהוא.
5. **Workers** — `Runtime.tick` + `automation.tick` כתהליך scheduler יחיד (נעילת Redis); מרוקנים `outbox` דרך אדפטרים אמיתיים; סודות בסביבת השרת בלבד channel workers.
6. **מעטפות מובייל** — PWA קודם, אחר כך חנויות. אותו bundle.

כל שלב ניתן לביצוע בנפרד ומשאיר את הקודם עובד. הגבולות שכבר קיימים בקוד (פונקציות `fn(db, actorId, …)`, `viewOf`) כחוזה מול הלקוח, מסכים שלא נוגעים ב-DB ישירות) הם מה שמאפשר את זה.

18. נספח: מפת קבצים, Git ושחזור

מפת `src/`

תוכן	תיקייה
`App.tsx` (נקודת כניסה + ראוטר ראשי), `session.ts` (ראוטים, session), `context.ts`, `hooks.ts` (`useDb`, `useT`, `go`), `shells.tsx` / `shells.v3.tsx`, `variants.ts` (בחירת v2/v3 לפי מעטפות), `design.ts`, `format.ts` (`__DESIGN__`)	`app/`
`cards.ts`, `rng.ts`, `variants.ts`, `holdem.ts`, `pots.ts`, `evaluator.ts`, `hands.ts`, `view.ts`, `session.ts`, `bots.ts`, `equity.ts`, `structure.ts`, `tournament.ts` — טהור, ללא DOM	`engine/`

תוכן	תיקייה
`types.ts` (הסכמה), `seed.ts`, `store.ts`, `lifecycle.ts`, `permissions.ts`, `core.ts`, `automation.ts`, `analytics.ts`, `crm.ts`, `api.ts`, `competition.ts`, `selectors.ts`, `messaging.ts`, `retention.ts`, `scheduling.ts`, `calendar.ts`, `channels.ts`, `integrity.ts`, `migration.ts`, `color.ts`, `table-branding.ts`, `runtime.ts`	`platform/`
מסכי השחקן (§8)	`player/`
`TableScreen.tsx`, `TableStage.tsx` / `.v3.tsx`, `ActionBar.tsx`, `Card.tsx`, `Chips.tsx`, `SpectatorScreen.tsx`, `layout.ts`, `sound.ts`, `text.tsx`	`table/`
קונסולת המועדון וה-CRM (§9–§10)	`console/` + `console/crm/`
`PlatformRouter.tsx`, `Overview`, `Clubs`, `Plans`, `Billing`, `Health`, `Support`, `Integrity` (§14)	`platform-admin/`
קומפוננטות בסיס (`core`, `forms`, `overlay`, `charts`), `identity`, `qr.ts` (מקודד QR עצמאי), `image.ts`	`ui/`
קומפוננטות משותפות לשחקן/קונסולה: `tournament(.v3).tsx`, `competition.tsx`, `qr.tsx`, `plans.tsx`, `support.tsx`, `weekdays.ts`	`shared/`
`en*.ts`, `he*.ts`, `index.ts`	`i18n/`
`app.css` (v2), `app-v3.css` (v3)	`styles/`

`scripts/`: `build.mjs` (esbuild → `dist/index.html`, 1.7~ קובץ יחיד MB), `dev.mjs` (watch + שרת סטטי), `e2e.mjs` (Playwright), `i18n-check.mjs`. `docs/`: `ARCHITECTURE.md`, `PRODUCTION_ARCHITECTURE.md`, `V04_FEATURE_AUDIT.md`, `V04_REPORT.md`, `DESIGN_V3.md`, `REDESIGN_REPORT_V3.md`, `CRM_AUDIT.md`, `COMPLETION_REPORT.md`, `screenshots/`.

קונבנציות קוד שחשוב לשמור:

- כל פונקציית שירות: `fn(db, actorId, ...args)`; הרשאות נבדקות בתחילת הפונקציה עם `requirePermission`, לא ב-UI.

- שינוי ל-`DB` רק דרך `store.mutate(fn)` — מעדכן `rev`, שומר ומעורר subscribers. אסור לשנות אובייקטים מחוץ לזה.
- מסכים לא קוראים `db.crm.*`, `accessCode` או `analytics.compute` ישירות (נאכף בבדיקה).
- זמן: תמיד `clock.now()` / פרמטר `now`, לעולם לא `Date.now()` ישירות בשכבת השירות (כך הבדיקות מזיזות זמן).
- אקראיות: `seededRng` בבדיקות וב-seed, `cryptoRng` במשחק חי.
- טקסט למשתמש — רק דרך `t()`; מפתח חדש נוסף ל-EN ול-HE יחד.
- הוספת שדה לסכמה = עדכון `types.ts` + `SCHEMA_VERSION` + ערך ברירת מחדל ב-`seed.ts`.

Git כ) נמסר המלא המקור ;(`poker-club-os-0.4.0-source.zip`-ריפו מקומי בסביבת העבודה

מה זה	ענף / תג
הגרסה הנוכחית — כל מה שמתואר במסמך זה	`feature/v0.4-features` @ `729ee4b`, תג `v0.4.0`
ללא מודולי Graphite & Ember עיצוב — v0.3 v0.4	`feature/v0.3-redesign`, תג `v0.3.0`, גיבוי `backup/v0.3-redesign`
העיצוב המקורי שלם	`backup/v0.2-original-design`, תג `v0.2.0`
לא עודכן מאז v0.2 — המיזוג ממתין לאישור	`main`

קומיטי v0.4 לפי אבן דרך: `399882e` מנוע וריאנטים · `6c04f5e` שילוב משחקים בטורנירים · `705fc1e` / `10bf631` תחרות · `0d51707` הפניות/QR/יומן · `3bed945` שימור · `8b015c2` / `c42984d` תפעול ותזמון · `3d5a315` צפייה/יושרה · `8fa2176` ערוצים/משחק אחראי/נגן · `55f44c1` QA · `adefd7c` תיעוד.

שחזור: `git checkout feature/v0.3-redesign` (או `git checkout v0.3.0`) מחזיר למצב לפני v0.4; `git checkout backup/v0.2-original-design` מחזיר לעיצוב המקורי. בכל מקרה — `node scripts/build.mjs --design=v3` (או `v2`) ופתיחת `dist/index.html`. שימו לב: `localStorage` של גרסה חדשה יותר (מפתח `pcos.db.v7`) לא נקרא על-ידי גרסה ישנה — היא תייצר seed משלה תחת מפתח משלה.